VERIQTA

TRIVY for DEVOPS BEGINNERS

From Zero to Container, Filesystem, IaC and CI/CD Security Scanning

CONTAINER IMAGES | FILESYSTEM SCANNING | IaC SCANNING | VULNERABILITY DETECTION | CI/CD INTEGRATION

FIND. PRIORITIZE. FIX. SECURE EVERY BUILD.

PRACTICAL HANDS-ON BEGINNER FRIENDLY

YouTube GitHub LinkedIn X Instagram Telegram | veriqta

1. WHAT IS TRIVY?

WHAT TRIVY IS

Trivy is a fast, simple and comprehensive open source security scanner from Aqua Security. It helps you find vulnerabilities, misconfigurations and secrets in your applications and infrastructure before they reach production.

WHY DEVOPS ENGINEERS USE TRIVY

- Integrates easily into CI/CD pipelines
- Scans early to reduce risk
- Prevents vulnerable code and images from going to production
- Open source, fast and lightweight
- Supports containers, code, IaC and more

WHAT SECURITY SCANNING MEANS

Security scanning is the process of automatically checking your code, containers, configurations and infrastructure for known security issues, policy violations and exposed secrets.

VULNERABILITY vs MISCONFIGURATION

VULNERABILITY

A weakness in software or libraries that can be exploited by an attacker.

Example:
Outdated OpenSSL with a known CVE.

MISCONFIGURATION

A security risk caused by incorrect or insecure configuration.

Example:
S3 bucket is public and allows anyone.

WHAT TRIVY CAN SCAN

CONTAINER IMAGES
Scan OS packages and application dependencies inside container images.

FILESYSTEM / CODE
Scan source code and project files for vulnerable dependencies and secrets.

IaC (INFRASTRUCTURE AS CODE)
Scan Terraform, CloudFormation, Kubernetes YAML, Dockerfile and more for misconfigurations.

VULNERABILITIES
Find known CVEs in OS packages, libraries and application components.

SECRETS
Detect exposed secrets, keys, tokens and credentials in code and files.

WHERE TRIVY FITS IN DEVOPS

Trivy fits everywhere security matters. The earlier you scan, the safer your delivery.

- Local development (shift-left)
- Pre-commit and pull requests
- CI/CD pipelines
- Before building container images
- Before deployment to any environment
- Continuous monitoring and compliance

SIMPLE FLOW: CODE → BUILD → SCAN → DEPLOY

CODE
Develop code, commit to repository.

→

BUILD
Build application and container images.

→

SCAN
Trivy scans for vulnerabilities, misconfigurations and secrets.

→

DEPLOY
Deploy safe artifacts to production.

KEY TAKEAWAY

Trivy helps DevOps teams find and fix security issues early in the development lifecycle. Scan early, scan often, and ship secure.

2. INSTALLING TRIVY AND RUNNING YOUR FIRST SCAN

1 INSTALLING TRIVY

Linux (using package manager)

```
$ sudo apt update
$ sudo apt install -y trivy
```

macOS (using Homebrew)

```
$ brew install aquasecurity/trivy/trivy
```

Windows (using Chocolatey)

```
$ choco install trivy
```

2 VERIFY THE INSTALLATION

Run the version command:

```
$ trivy --version
Version: 0.50.0
Vulnerability DB:
Updated: 2024-05-20 12:00:00 +0000
Next Update: 2024-05-20 18:00:00 +0000
```

If you see version information, Trivy is installed correctly!

3 UNDERSTANDING THE CLI STRUCTURE

```
trivy [command] [target] [options]
```

- `command` : What you want Trivy to do (image, fs, config, etc.)
- `target` : What to scan (image name, directory, file, etc.)
- `options` : How to run the scan (severity, format, output, etc.)

Common commands:

image	fs	config	secret
Scan container images	Scan filesystems and projects	Scan IaC files (Terraform, K8s, etc.)	Scan for exposed secrets

4 RUN YOUR FIRST SCAN (IMAGE)

Scan the official Nginx image:

```
$ trivy image nginx:latest
```

5 READING YOUR FIRST RESULT

Trivy shows vulnerabilities in packages and OS libraries.

SAMPLE OUTPUT (SHORTENED)

TARGET	TYPE	VULNERABILITY ID	SEVERITY	PACKAGE	INSTALLED	FIXED IN
nginx:latest (alpine 3.19)	os	CVE-2024-21626	HIGH	openssl	3.1.2-r0	3.1.2-r1
nginx:latest (alpine 3.19)	os	CVE-2024-23334	CRITICAL	busybox	1.36.1-r12	1.36.1-r13
nginx:latest (alpine 3.19)	os	CVE-2024-3094	HIGH	zlib	1.2.13-r1	1.2.13-r2

Each finding tells you what is vulnerable, how severe it is, and which version fixes it.

6 WHAT HAPPENS WHEN TRIVY SCANS?

- Trivy identifies the target (image, filesystem, IaC, etc.).
- Trivy collects package and dependency information.
- Trivy checks against the vulnerability database.
- Trivy matches known vulnerabilities.
- Trivy reports severity, affected package and fix version.
- You review, remediate, rebuild and rescan.

7 FIRST SCAN WORKFLOW

Install Trivy → Verify Installation → Run Scan → Read Results → Fix & Rescan

QUICK TIP

Start by scanning official images like nginx:latest to understand the output. Then scan your own images and code.

VERIQTA

3. UNDERSTANDING VULNERABILITIES BEFORE SCANNING

1 WHAT IS A CVE?

A CVE (Common Vulnerabilities and Exposures) is a unique identifier for a publicly known security vulnerability.

CVE helps everyone refer to the same vulnerability in the same way.

2 CVE IDENTIFIERS

CVE-2024-12345

Prefix	Year	Unique ID
Always CVE	Year the CVE was published	Unique number for the vulnerability

VULNERABILITY DATABASES

Trivy uses multiple sources to find vulnerabilities, including:

- NVD (National Vulnerability Database)
- Red Hat / Ubuntu / Alpine Advisories
- GitHub Advisory Database
- Vendor Security Advisories
- CISA Known Exploited Vulnerabilities (KEV)

4 SEVERITY LEVELS

UNKNOWN	LOW	MEDIUM	HIGH	CRITICAL
Not enough information to determine the severity.	Low risk. Unlikely to be exploited or has minimal impact.	Moderate risk. Could be exploited with some effort.	High risk. Likely to be exploited and can impact systems.	Very high risk. Easily exploited with severe impact.

LOWER RISK → HIGHER RISK

5 CVSS BASICS

CVSS (Common Vulnerability Scoring System) measures the severity of a vulnerability.

- Score range: 0.0 to 10.0
- 0.0 = No risk
- 10.0 = Maximum risk
- Higher score = Higher priority

CVSS SCORE GUIDE

0.0	0.1 – 3.9	4.0 – 6.9	7.0 – 8.9	9.0 – 10.0
NONE	LOW	MEDIUM	HIGH	CRITICAL

6 FIXED vs UNFIXED VULNERABILITIES

FIXED

A patched version is available.

You can update the package or image to remove the risk.

UNFIXED

No patch is available yet.

You must apply workarounds or monitor the risk.

7 WHY NOT EVERY CVE REQUIRES THE SAME RESPONSE?

NOT ALL VULNERABILITIES ARE EXPLOITABLE

Some require very specific conditions that may not exist in your environment.

ATTACK COMPLEXITY MATTERS

Some CVEs are easy to exploit. Others need high skill or access.

IMPACT VARIES

Not every vulnerability can cause data loss, system takeover or business impact.

COMPENSATING CONTROLS EXIST

Other security controls may already reduce or eliminate the risk.

RISK BASED PRIORITIZATION

Focus on CVEs that are exploitable, high impact and relevant to your environment.

FOCUS YOUR EFFORT WHERE IT MATTERS MOST:
Look at severity, exploitability, impact, and availability of a fix before taking action.

4. CONTAINER IMAGE SCANNING

USE TRIVY TO FIND VULNERABILITIES IN YOUR CONTAINER IMAGES BEFORE THEY REACH PRODUCTION.

trivy image

Scan a Docker image for vulnerabilities in OS packages and application dependencies.

BASIC COMMAND

```
trivy image myapp:1.0
```

FILTER EXAMPLE (HIGH & CRITICAL ONLY)

```
trivy image --severity HIGH,CRITICAL myapp:1.0
```

WHAT TRIVY SCANS IN AN IMAGE

- **Image Layers** — Trivy analyzes every layer in the image.
- **Application Dependencies** — Finds vulnerabilities in libraries and frameworks.
- **OS Package Vulnerabilities** — Detects issues in the operating system packages.
- **Base OS** — Checks the base image for known vulnerabilities.

KEY CAPABILITIES

Scan a Docker Image
Scan local or remote images from any registry.

Image Layers
Trivy inspects all layers, not just the top layer.

OS Package Vulnerabilities
Detects known CVEs in installed OS packages.

Application Dependency Vulnerabilities
Finds vulnerable libraries and modules.

Severity Filtering
Filter results by severity to focus on what matters most.

Understand Scan Results
Clear and actionable output to prioritize and fix issues.

UNDERSTANDING SCAN RESULTS

TARGET	PACKAGE / LIBRARY	VULNERABILITY ID	SEVERITY	INSTALLED VERSION	FIXED VERSION	TITLE
Image or layer that was scanned.	Affected OS package or application library.	CVE identifier (e.g., CVE-2023-12345).	CRITICAL HIGH MEDIUM LOW UNKNOWN	Vulnerable version installed in the image.	Version that contains the fix.	Short description of the vulnerability.

WHY IT MATTERS

- Prevent vulnerable images from reaching production.
- Reduce risk from known CVEs.
- Keep your base images and dependencies secure.
- Improve compliance and security posture.

EXAMPLE

```
$ trivy image --severity HIGH,CRITICAL myapp:1.0
Total: 8 (HIGH: 6, CRITICAL: 2)
...
myapp:1.0 (alpine 3.18)
├── CRITICAL: CVE-2023-12345
├── HIGH: CVE-2023-54321
└── HIGH: CVE-2023-67890
```

5. READING TRIVY IMAGE SCAN RESULTS

UNDERSTANDING EACH COLUMN

TARGET
The scanned image layer or component.

PACKAGE / LIBRARY
The OS package or application library affected.

VULNERABILITY ID
The CVE identifier for the vulnerability.

SEVERITY
The risk level of the vulnerability (Low to Critical).

INSTALLED VERSION
The version currently present in the image.

FIXED VERSION
The version that resolves the issue.

VULNERABILITY TITLE
A short description of what the vulnerability is.

EXAMPLE: TRIVY IMAGE SCAN OUTPUT (TABLE VIEW)

TARGET	PACKAGE / LIBRARY	VULNERABILITY ID	SEVERITY	INSTALLED VERSION	FIXED VERSION	VULNERABILITY TITLE
alpine:3.19	libssl3	CVE-2023-5678	CRITICAL	3.1.1-r1	3.1.2-r0	OpenSSL out-of-bounds read vulnerability
alpine:3.19	busybox	CVE-2023-45853	HIGH	1.36.1-r2	1.36.1-r3	BusyBox heap-based buffer overflow
alpine:3.19	curl	CVE-2023-38545	MEDIUM	8.4.0-r0	8.4.0-r1	libcurl DoS vulnerability
alpine:3.19	zlib	CVE-2022-37434	LOW	1.2.12-r3	1.2.13-r0	Zlib information disclosure

HOW TO DECIDE WHAT NEEDS FIXING

- Focus on HIGH and CRITICAL first.
- Check if a fix version exists.
- Consider exploitability and exposure.
- Low severity may be acceptable if no fix exists or not exploitable.
- Always review the vulnerability title and context before taking action.

BEGINNER WORKFLOW FOR INVESTIGATING FINDINGS

1. **IDENTIFY** — Find HIGH and CRITICAL issues in the results.
2. **RESEARCH** — Look up the CVE ID for details (NVD, GitHub, Vendor site).
3. **ASSESS RISK** — Check the severity, exploit availability and impact on your application.
4. **FIND FIX** — Check the fixed version or vendor advisory for remediation steps.
5. **UPDATE** — Update the package, library or base image to a safe version.
6. **RESCAN** — Re-run Trivy to confirm the issue is resolved.

TIP: Always keep your base images, packages and dependencies up to date. Regular scanning helps you catch and fix issues before they reach production.

6. FIXING VULNERABLE CONTAINER IMAGES

Fixing vulnerabilities is not just about scanning.
It's about taking action and rebuilding your image safely.

1. UPDATING BASE IMAGES

- Use official, minimal base images.
- Pull the latest secure tag or digest.
- Example: python:3.12-slim instead of old tags.

2. UPDATING PACKAGES

- Keep OS packages up to date.
- Use package manager update/upgrade.
- Example: apt-get update && apt-get upgrade

3. UPDATING DEPENDENCIES

- Update libraries and dependencies.
- Use trusted, compatible versions.
- Example: npm update or pip install --upgrade

4. PINNING SAFER VERSIONS

- Pin specific versions in Dockerfile or lock files.
- Prevents unexpected updates.
- Example: package@1.2.3 instead of latest.

5. REBUILDING THE IMAGE

- Rebuild the image after all fixes.
- Old layers may still contain vulnerabilities.
- Always build a new clean image.

6. RESCANNING AFTER REMEDIATION

- Run Trivy again on the new image.
- Ensure HIGH and CRITICAL issues are fixed.
- Confirm the image is safe to deploy.

7. WHY REBUILDING MATTERS

- Vulnerabilities can exist in old layers.
- Updating without rebuilding does not remove them.
- Rebuild ensures all changes are applied correctly.

THE SECURE IMAGE REMEDIATION CYCLE

SCAN
Scan the image and identify vulnerabilities.

FIX
Update base image, packages, dependencies and configurations.

REBUILD
Rebuild the container image to apply all changes and remove old vulnerable layers.

RESCAN
Rescan the new image to verify vulnerabilities are resolved.

REMEMBER

Security is an ongoing process.
Scan early, fix completely, rebuild always, and rescan to be sure.

7. FILESYSTEM SCANNING

trivy fs • Scan Your Code Before You Build

WHAT IS trivy fs?

trivy fs scans your local files, dependencies and configuration before building a container image. It helps you find vulnerabilities, misconfigurations and secrets early in the development cycle.

SCAN A PROJECT DIRECTORY

```
$ trivy fs .
```

This command scans the current directory recursively and analyzes all supported files.

WHAT FILESYSTEM SCANNING CHECKS

DEPENDENCY FILES

Scans language dependency files like package.json, requirements.txt, pom.xml, go.mod, Gemfile and more.

LOCK FILES

Scans lock files such as package-lock.json, Pipfile.lock, yarn.lock, Gemfile.lock, Cargo.lock to find vulnerable resolved versions.

LOCAL SOURCE REPOSITORIES

Scans your source code for issues, misconfigurations and insecure settings.

CONFIGURATION FILES

Scans IaC and config files (Dockerfile, Terraform, Kubernetes manifests, etc.) for security problems.

SECRETS

Detects hardcoded secrets, API keys, tokens, passwords and other sensitive information.

IMAGE SCANNING vs FILESYSTEM SCANNING

IMAGE SCANNING (trivy image)

- Scans built container images
- Analyzes OS packages and layers
- Finds vulnerabilities in the final image
- Used after building the image
- Focuses on runtime environment

VS

FILESYSTEM SCANNING (trivy fs)

- Scans your local files and code
- Finds issues before building
- Catches problems earlier (shift-left)
- Checks dependencies, configs, secrets
- Helps prevent bad images from being built

WHEN DEVOPS ENGINEERS USE EACH

USE trivy fs WHEN:

- Writing code locally
- Before committing or pushing code
- During pull request validation
- Before building container images
- Scanning IaC or config files in repos

USE trivy image WHEN:

- After building container images
- Before pushing images to registry
- Before deploying to environments
- In CI/CD after image build step
- For runtime environment assurance

THE SECURE DEVELOPMENT FLOW

SCAN → FIX → REBUILD → RESCAN

SCAN → FIX → REBUILD → RESCAN

Repeat until there are no unacceptable risks.
Build secure. Ship secure.

8. REPOSITORY SCANNING

SCAN EARLY. FIND ISSUES BEFORE YOU BUILD.

SCANNING SOURCE REPOSITORIES

- Trivy can scan your source code before containers are built.
- Detects vulnerabilities, secrets and misconfigurations early.
- Stops problems from reaching production.

DEPENDENCIES BEFORE CONTAINER BUILDS

- Find vulnerable libraries and packages in your project files.
- Catch issues in dependency and lock files.
- Prevents vulnerable images from being built.

FINDING PROBLEMS EARLIER

- Cheaper and faster to fix.
- Reduces rework and re-runs.
- Improves code quality and team productivity.
- Build secure by default.

LOCAL REPOSITORY WORKFLOW

1. CLONE CODE
Get the repository to your machine.

2. RUN TRIVY FS
Scan the project directory.

3. REVIEW RESULTS
Review vulnerabilities, secrets and misconfigurations.

4. FIX ISSUES
Update dependencies, remove secrets, fix misconfigurations.

5. RESCAN
Run the scan again to confirm everything is fixed.

REPOSITORY SCANNING IN DEVELOPMENT

- Run Trivy scans on every commit or PR.
- Block risky code before merge.
- Use in IDE, pre-commit hooks and CI.
- Works for developers, security and DevOps.
- Makes security a part of daily development.

FILESYSTEM vs REPOSITORY SCANNING

FEATURE	FILESYSTEM SCANNING	REPOSITORY SCANNING
WHAT IT SCANS	Local files and directories	Project source repository
FOCUS	Dependencies, secrets, IaC, configs	Code + dependencies + configs + secrets
USE CASE	Local development checks	Continuous development and code review
WHEN TO USE	Before build or at any time on local files	Every commit, PR and in CI/CD

SHIFT-LEFT SECURITY EXPLAINED SIMPLY

Security used to happen at the end (after build or release).
That is "shift-right" and it is expensive.

Shift-left means moving security to the beginning of the development cycle.
Scan early. Fix early. Ship secure.

KEY TAKEAWAY Scanning your repository early helps you find and fix problems before they become big security risks in production.

9. INFRASTRUCTURE AS CODE SCANNING

Secure your infrastructure before you deploy it.

WHAT IaC SECURITY SCANNING MEANS

IaC (Infrastructure as Code) scanning checks your infrastructure definitions for security risks, policy violations and dangerous configurations before they reach production.

WHY VALID CONFIGURATION CAN STILL BE INSECURE

Your IaC can be syntactically correct (valid) but still contain security risks such as:

- Public exposure
- Over-permissive access
- Weak encryption
- Missing security controls
- Unrestricted network rules

Trivy SUPPORTS MULTIPLE IaC TYPES

TERRAFORM
.tf, .hcl

KUBERNETES MANIFESTS
YAML files

DOCKERFILES
Dockerfile

CLOUDFORMATION
.yaml / .yml / JSON

MISCONFIGURATION DETECTION

Trivy identifies security issues such as:

- Public S3 buckets
- Open security groups (0.0.0.0/0)
- Root user or privileged containers
- Unencrypted storage or databases
- Hardcoded secrets or credentials
- Missing logging and monitoring
- Insecure IAM permissions
- Best practice violations

EXAMPLE: INSECURE INFRASTRUCTURE WORKFLOW

1. WRITE IaC
Developer writes Terraform / YAML / CloudFormation.

→

2. VALIDATE SYNTAX
IaC is syntactically correct. Validation passes.

→

3. DEPLOY
Infrastructure is deployed to AWS or Kubernetes.

→

4. RISK EXISTS
Misconfigurations create security exposures.

→

5. DETECT & FIX
Trivy IaC scan finds issues. Fix, commit and rescan.

KEY TAKEAWAY

Infrastructure code can be valid but still be dangerous. Scan early, fix early, deploy safely.

VERIQTA

10. SCANNING TERRAFORM WITH TRIVY

SCAN YOUR IaC. FIND RISKS BEFORE DEPLOYMENT.

TERRAFORM SECURITY BASICS

Terraform code can create powerful cloud resources. Misconfigurations can lead to serious security risks.

Trivy helps you detect insecure patterns before they reach production.

COMMON IaC RISKS TRIVY DETECTS

PUBLIC EXPOSURE
Resources open to the internet without restrictions.

ENCRYPTION PROBLEMS
Missing encryption for data at rest or in transit.

INSECURE NETWORK RULES
Overly permissive security groups, NACLs or open ports.

LOGGING CONFIGURATION
Missing or insufficient logging and monitoring settings.

RUNNING A TERRAFORM SCAN

```
# Scan the current Terraform directory
trivy config .

# Scan a specific Terraform file or folder
trivy config ./terraform

# Output results in JSON
trivy config . -f json -o trivy-terraform.json

# Fail the scan on HIGH and CRITICAL
trivy config . --severity HIGH,CRITICAL --exit-code 1
```

Trivy analyzes your Terraform code for security misconfigurations.

UNDERSTANDING MISCONFIGURATION FINDINGS

SEVERITY	CHECK ID	WHAT IT MEANS	EXAMPLE
HIGH	AVD-AWS-0129	S3 bucket is public	Bucket allows public read access
MEDIUM	AVD-AWS-0022	Unencrypted S3 bucket	No encryption at rest
LOW	AVD-AWS-0037	Logging not enabled	CloudTrail not enabled
INFO	AVD-AWS-0057	Security group too open	0.0.0.0/0 ingress rule

Always read the description and remediation guidance provided by Trivy.

WHY SCANNING IaC IS CRITICAL

- PREVENT MISCONFIGURATIONS BEFORE DEPLOYMENT
- REDUCE CLOUD SECURITY RISKS
- AVOID COSTLY INCIDENTS
- ENSURE COMPLIANCE AND BEST PRACTICES

THE IaC SECURITY WORKFLOW

1. FIX
Review the findings and fix insecure configurations.

→

2. RESCAN
Run Trivy again to verify the issues are resolved.

→

3. COMMIT
Commit the secure Terraform code to your repository.

→

4. READY TO APPLY
Your IaC is now safer to apply to any environment.

11. SCANNING KUBERNETES MANIFESTS

SECURE YOUR KUBERNETES DEPLOYMENTS BEFORE THEY REACH THE CLUSTER

KUBERNETES YAML SCANNING

Trivy scans Kubernetes manifests (YAML files) to find misconfigurations and insecure settings that can lead to security risks in production.

Think of it as a security check for your Kubernetes configurations.

COMMAND

```
$ trivy k8s <file-or-directory>
```

Example:

```
$ trivy k8s .
```

WHAT TRIVY CHECKS IN KUBERNETES MANIFESTS

PRIVILEGED CONTAINERS

Detects containers running with privileged: true which gives excessive access to the host.

RUNNING AS ROOT

Identifies containers running as root user (runAsUser: 0) which increases risk if the container is compromised.

DANGEROUS CAPABILITIES

Finds containers adding dangerous Linux capabilities like NET_ADMIN, SYS_ADMIN, SYS_MODULE, etc.

MISSING SECURITY CONTROLS

Checks for missing security controls like readOnlyRootFS, allowPrivilegeEscalation, seccompProfile, AppArmor, etc.

RESOURCE CONFIGURATION

Detects missing or weak resource limits and requests that can cause instability or DoS situations.

SECRETS AND CONFIGURATION RISKS

Detects secrets in manifests, config risks and sensitive data exposure.

EXAMPLE RISKS TRIVY CAN FIND

- privileged: true
- runAsUser: 0 (root user)
- allowPrivilegeEscalation: true
- hostNetwork: true
- hostPID: true
- hostIPC: true
- dangerous capabilities (e.g. NET_ADMIN)
- missing readOnlyRootFilesystem
- secrets hardcoded in YAML
- missing resource limits

EXAMPLE INSECURE KUBERNETES YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: insecure-pod
spec:
  containers:
  - name: app
    image: myapp:latest
    securityContext: {}   # runs as root
    privileged: true
    capabilities:
      add:
        - NET_ADMIN          # dangerous capability
    resources: {}            # no limits or requests
```

WHY FIXING FINDINGS BEFORE DEPLOYMENT MATTERS

- Prevents security breaches
- Avoids runtime failures
- Meets security best practices
- Protects cluster, data and users
- Reduces incident response cost

FIX FINDINGS IN YOUR MANIFESTS, THEN DEPLOY WITH CONFIDENCE!

12. SCANNING DOCKERFILES

Scan early. Build safe. Ship secure.

WHY DOCKERFILES SHOULD BE SCANNED

- Dockerfiles define how your image is built.
- A small mistake can introduce serious security risks.
- Trivy detects misconfigurations and vulnerabilities in your Dockerfile before you build.
- Scan early to prevent insecure images from reaching production.

KEY RISKS TRIVY CHECKS

BASE IMAGE RISKS
Using outdated or untrusted base images with known vulnerabilities.

ROOT CONTAINERS
Running as root increases the impact of a potential compromise.

PACKAGE INSTALLATION PRACTICES
Unsafe package managers, no version pinning, or unverified packages.

SECRETS IN BUILD CONFIGURATION
Hardcoded credentials, tokens or secrets in Dockerfile instructions.

MISCONFIGURATION FINDINGS TRIVY CAN DETECT

- Using latest tag without version pinning
- Running as root USER not specified
- Installing unnecessary packages or using risky package managers
- Secrets or sensitive data in ENV, ARG, ADD, COPY or RUN instructions
- Using ADD instead of COPY for normal files
- Missing security best practices (least privilege, small base image)

BUILDING SAFER CONTAINER IMAGES

- Use minimal and trusted base images
- Pin image versions explicitly
- Run as non-root user
- Install only required packages
- Remove package caches and unnecessary files
- Never store secrets in Dockerfiles
- Scan before every build

DOCKERFILE EXAMPLE (BETTER PRACTICE)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production \
    && npm cache clean --force
COPY . .
USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

TIP

SMALLER IMAGE.
LESS SURFACE.
LOWER RISK.
BETTER SECURITY.

SECURE DOCKERFILE WORKFLOW

1. DOCKERFILE
Write or update your Dockerfile.

2. SCAN
Scan the Dockerfile using Trivy.
`trivy config Dockerfile`

3. BUILD
Build the container image after fixing issues.
`docker build -t myapp:1.0 .`

4. IMAGE SCAN
Scan the built image for vulnerabilities.
`trivy image myapp:1.0`

SCAN EARLY. FIX FAST. BUILD SAFE. SHIP SECURE.
SECURITY IS A PROCESS, NOT AN AFTERTHOUGHT.

VERIQTA

13. SECRET SCANNING

FIND AND PROTECT SENSITIVE SECRETS BEFORE THEY CAUSE DAMAGE

WHAT SECRET SCANNING DETECTS

API KEYS
Keys used to access external services.

TOKENS
Authentication tokens, bearer tokens, and access tokens.

CREDENTIALS
Usernames, passwords and service account credentials.

PRIVATE KEYS
SSH private keys, RSA, EC, PGP and other private keys.

AND MORE
Cloud keys, database URLs, secrets in code, config files and environments.

WHY SECRETS ENTER GIT REPOSITORIES

- Developers accidentally commit secrets.
- Copying from docs, Slack, emails or environments.
- Hardcoding credentials in code or config.
- Secrets in test files, examples or templates.
- Lack of secret management practices.
- Unaware that the file is tracked by Git.
- Shared repositories and forks.

SCAN FOR SECRETS WITH TRIVY

```
trivy fs --scanners secret .
```

Scans the current directory for exposed secrets.
Works on any project or repository.

WHAT TO DO AFTER FINDING AN EXPOSED SECRET

1. **IDENTIFY THE SECRET**
   Understand what secret was exposed and where it appeared.
2. **REVOKE OR ROTATE IMMEDIATELY**
   Generate a new key, token or credential. Invalidate the old one.
3. **REMOVE FROM CODE AND HISTORY**
   Delete the secret from files and remove it from Git history using git filter-repo or BFG Repo-Cleaner.
4. **COMMIT CLEAN CHANGES**
   Commit the cleaned code and push to the repository.
5. **NOTIFY AND REVIEW ACCESS**
   Inform your team. Review where the secret was used. Check for abuse or unauthorized access.
6. **PREVENT FUTURE LEAKS**
   Use secret managers (AWS Secrets Manager, Vault, etc.), add Trivy secret scans in CI/CD, and enable pre-commit checks.

WHY DELETING THE FILE ALONE IS NOT ENOUGH

- ✗ The secret still exists in Git history.
- ✗ Anyone with repo access can recover it.
- ✗ Search engines and bots may have already indexed it.
- ✗ Attackers may have already captured and used it.
- ✗ The secret remains in clones, forks and backups.
- ✗ Compliance and security risks remain until fully removed and rotated.

REMEMBER
A secret is only safe when it is revoked, removed from history, and protected in the future.

EXAMPLE WORKFLOW

1. TRIVY FINDS A SECRET → 2. ROTATE / REVOKE THE SECRET → 3. REMOVE FROM CODE AND HISTORY → 4. PUSH CLEAN CHANGES → 5. PREVENT AND MONITOR

14. CONTROLLING WHAT TRIVY SCANS

1. SCANNER SELECTION

- Choose the scanners you need for your workflow.

```
# Enable only what you need
trivy fs --scanners vuln,secret,config .
```

2. SEVERITY FILTERS

- Focus on the issues that matter most.

```
# Scan only High and Critical
trivy image --severity HIGH,CRITICAL myapp:1.0
```

3. IGNORING UNFIXED VULNERABILITIES

- Hide vulnerabilities that have no available fix yet.

```
trivy fs --ignore-unfixed .
```

4. SKIP FILES AND DIRECTORIES

- Exclude paths that are not relevant.

```
trivy fs --skip-dirs node_modules,testdata .
```

5. .TRIVYIGNORE

.trivyignore

- Create a .trivyignore file to permanently exclide, directories or vulnerabilities.

```
# Example .trivyignore
node_modules/
testdata/
*.md
CVE-2021-12345
docker-images/*
```

SUPPORTED PATTERNS:

- File and directory paths
- Wildcard patterns
- Specific CVE IDs
- Docker image patterns

6. WHEN IGNORING MAY BE JUSTIFIED

- No fix is available and risk is accepted.
- False positive confirmed.
- Vulnerability is not reachable in your environment.
- Risk is mitigated by compensating controls.
- Temporary exception with a clear review date.

7. WHY BLINDLY SUPPRESSING FINDINGS IS DANGEROUS

- Hides real security risks.
- Leads to false sense of security.
- Attackers exploit ignored vulnerabilities.
- Compliance and audits can fail.
- Harder to track and manage real issues.

8. KEEPING SCAN RESULTS USEFUL

- Scan with purpose. Enable only what you need.
- Use severity filters to focus on real risks.
- Ignore wisely, not permanently or blindly.
- Review ignored items regularly.
- Document the reason for every exception.
- Keep your scans clean, relevant and actionable.

QUICK COMMAND REFERENCE

`trivy fs .`	`trivy fs --scanners vuln,secret,config .`	`trivy fs --severity HIGH,CRITICAL .`	`trivy fs --ignore-unfixed .`	`trivy fs --skip-dirs dir1,dir2 .`	`trivy fs --help`
Scan current directory	Select specific scanners	Filter by severity	Ignore unfixed vulnerabilities	Skip directories	See all available options

CONTROL YOUR SCANS SMARTLY. FIND WHAT MATTERS. IGNORE WHAT DOESN'T. KEEP YOUR SECURITY SIGNAL STRONG AND ACTIONABLE.

15. TRIVY OUTPUT AND REPORTS

TRIVY CAN PRODUCE MULTIPLE OUTPUT FORMATS FOR HUMANS AND MACHINES.

1. TABLE OUTPUT

- Default, human-readable
- Easy to read in terminal
- Good for quick reviews
- Shows key details

```
$ trivy image nginx:latest
```

LIBRARY	VULNERABILITY	SEVERITY
openssl	CVE-2023-0464	HIGH
zlib	CVE-2023-45853	MEDIUM
...	...	...

2. JSON OUTPUT

- Machine-readable
- Easy to parse with tools
- Ideal for automation
- Full structured data

```
$ trivy image nginx:latest -f json
{
  "Results": [
    {
      "Target": "nginx:latest",
      "Vulnerabilities": [
        {
          "VulnerabilityID": "CVE-2023-0464",
          "PkgName": "openssl",
          "Severity": "HIGH"
        }
      ]
    }
  ]
}
```

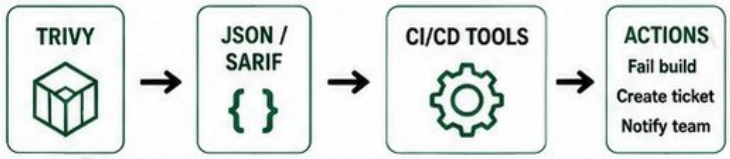
3. SARIF OUTPUT

- Industry standard for security tools
- Integrates with GitHub, GitLab, Azure DevOps and more
- Shows results in code scanning dashboards

```
$ trivy fs . -f sarif -o trivy.sarif
{
  "version": "2.1.0",
  "runs": [ ... ],
  "results": [ ... ]
}
```

4. MACHINE-READABLE RESULTS

- JSON and SARIF are designed for automation
- Feed results into dashboards, tickets, notifications and policies
- Use exit codes to fail builds on HIGH/CRITICAL findings



5. SAVING REPORTS

- Save output to files for later review
- Share with teams or auditors
- Keep history of scans

```
$ trivy image myapp:1.0 -f json -o report.json
$ trivy fs . -f sarif -o report.sarif
$ trivy image myapp:1.0 -o report.txt
```

6. HUMAN REVIEW vs AUTOMATION

HUMAN REVIEW

- Understand context
- Validate findings
- Decide what to fix
- Reduce false positives

VS

AUTOMATION

- Enforce policies
- Block risky builds
- Create tickets
- Provide fast feedback

7. USING REPORTS IN SECURITY WORKFLOWS

8. CHOOSING THE RIGHT OUTPUT FOR CI/CD

USE CASE	BEST OUTPUT	WHY?
Quick manual review in terminal	TABLE (Default)	Easy to read and understand
Automation and custom tooling	JSON	Structured data for programs
Code scanning integration (GitHub, GitLab, Azure DevOps)	SARIF	Native support in code scanning dashboards
Compliance, audit and reporting	JSON or SARIF	Machine-readable and exportable
Simple logs and artfacts	TABLE or TEXT	Lightweight and readable

KEY TAKEAWAY

Use the right output for the right job.
Humans read tables.
Machines read JSON and SARIF.
Automate the boring. Focus on the risk.

16. USING TRIVY IN CI/CD

WHERE SECURITY SCANNING BELONGS

Security scanning must be built into the CI/CD pipeline at multiple stages to catch issues early and prevent vulnerable code or images from reaching production.

OUR GOAL

FAIL UNSAFE BUILDS BEFORE PRODUCTION

TRIVY IN THE CI/CD PIPELINE

1. **SOURCE SCAN** — Scan your source code for secrets, hardcoded credentials and sensitive information.
2. **DEPENDENCY SCAN** — Scan dependency files and lock files for vulnerabilities in libraries and packages.
3. **IaC SCAN** — Scan infrastructure as code (Terraform, CloudFormation, Kubernetes YAML) for misconfigurations and risks.
4. **CONTAINER BUILD** — Build your container image with the latest code and configurations.
5. **IMAGE SCAN** — Scan the built container image for OS package and application vulnerabilities.
6. **DEPLOYMENT GATE** — Block the pipeline if HIGH or CRITICAL vulnerabilities or misconfigurations are found.
7. **DEPLOY** — Deploy only secure and approved artifacts to production.

WHY THIS MATTERS

- Find issues early when they are cheaper and easier to fix.
- Stop vulnerable code, misconfigurations and images from reaching production.
- Improve security posture without slowing down development.
- Keep your applications, infrastructure and data safe.

NEVER DEPLOY UNSAFE BUILDS!

One vulnerable build can become a production incident.

TRIVY CI/CD BENEFITS

- AUTOMATED SECURITY
- EARLY RISK DETECTION
- REDUCED COSTS
- BETTER TEAM COLLABORATION
- COMPLIANCE AND AUDIT READY
- SAFER RELEASES TO PRODUCTION

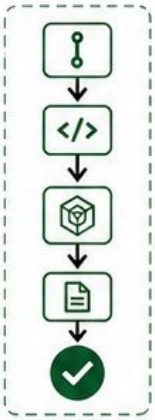
THE GOLDEN RULE

SCAN → FIX → REBUILD → RESCAN → REPEAT UNTIL CLEAN

17. TRIVY WITH GITHUB ACTIONS

1. BASIC WORKFLOW STRUCTURE

1. Trigger (PR / Push)
2. Checkout Code
3. Run Trivy Scan
4. Upload Results
5. Fail or Pass
6. Notify / Summary



2. TRIGGERING SCANS ON PULL REQUESTS

- Run scans automatically on Pull Requests.
- Catch issues before code is merged.
- Protect your main branch.
- Example trigger:

```
on:
  pull_request:
    branches: [ main ]
```

3. SCANNING CONTAINER IMAGES

- Build image and scan it.
- Detect OS packages and application vulnerabilities.
- Example command:

```
trivy image --exit-code 1 --severity HIGH,CRITICAL myapp:latest
```

4. SCANNING REPOSITORIES

- Scan source code, dependencies and configurations.
- Detect vulnerable libraries and secrets.
- Example command:

```
trivy fs --exit-code 1 --severity HIGH,CRITICAL .
```

5. SEVERITY THRESHOLDS

- Filter by severity to reduce noise.
- Common levels: LOW, MEDIUM, HIGH, CRITICAL.
- Example:

```
--severity HIGH,CRITICAL
```

6. EXIT CODES

- trivy returns exit codes to control workflow.
- 0 = No issues (or only ignored issues)
- 1 = Vulnerabilities found (based on threshold)
- Other codes = Errors

7. FAILING A WORKFLOW SAFELY

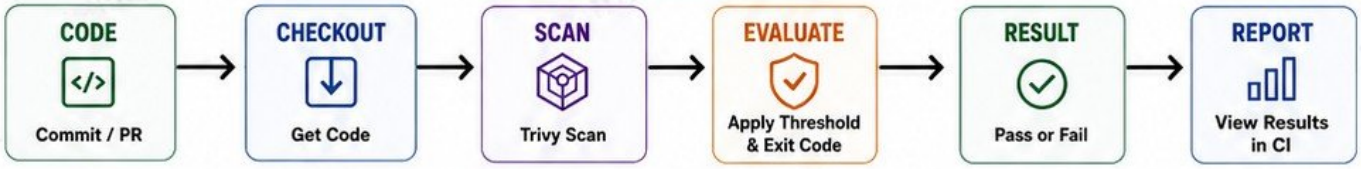
- Use --exit-code 1 to fail builds when HIGH/CRITICAL issues exist.
- Prevents insecure code or images from being merged or deployed.
- Safe by default, secure by design.

8. READING SECURITY RESULTS FROM CI

- Trivy can output results in different formats.
- Use SARIF to see results directly in GitHub Security tab.
- Review results in workflow summary or artifacts.
- Example output options:

```
trivy fs --format sarif -o trivy-results.sarif .
trivy image --format sarif -o trivy-results.sarif myapp:latest
```

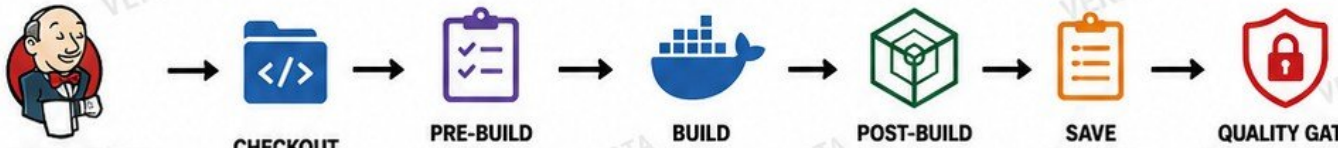
CI SECURITY FLOW WITH TRIVY



18. TRIVY WITH JENKINS

ADD SECURITY SCANNING TO YOUR PIPELINE. FAIL UNSAFE BUILDS BEFORE THEY SHIP.

ADDING TRIVY TO A JENKINS PIPELINE



SCAN BEFORE IMAGE CREATION (TRIVY FS)

- Scans your source code, dependencies, lock files, IaC and secrets before building.
- Catches issues early in the development cycle.
- Example command:

```
trivy fs . --format table --exit-code 1 \
--severity HIGH,CRITICAL
```

SCAN AFTER IMAGE CREATION (TRIVY IMAGE)

- Scans the built container image for OS packages and application dependencies.
- Ensures the final image is free from known vulnerabilities.
- Example command:

```
trivy image myapp:1.0 --format table --exit-code 1 \
--severity HIGH,CRITICAL
```

USING EXIT CODES

Trivy uses exit codes to control pipeline flow.

- 0 = No vulnerabilities above the threshold
- 1 = Vulnerabilities found (PIPELINE FAILS)
- 2 = Error running Trivy (PIPELINE FAILS)

Use `--exit-code 1` to fail the build on risk.

BLOCKING VULNERABLE BUILDS

- Fail the build automatically when HIGH or CRITICAL vulnerabilities are found.
- Prevents unsafe images from being pushed and deployed.
- Protects your environments by default.

SAVING SCAN REPORTS

- Store Trivy reports as build artifacts.
- Use formats like table, json, sarif, or html.
- Helps with audits, tracking and sharing findings.
- Example:

```
trivy image myapp:1.0 -f json
-o trivy-image-report.json
```

PIPELINE FAILURE BEHAVIOR

- If Trivy finds issues (exit code 1), the pipeline stops immediately.
- No image push.
- No deployment.
- Team is notified.
- Fix → Commit → Pipeline passes.

PRACTICAL JENKINS SECURITY GATE (EXAMPLE PIPELINE)

```
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Pre-Build Scan (Trivy FS)') {
      steps { sh 'trivy fs . --exit-code 1 --severity HIGH,CRITICAL' }
    }
    stage('Build Image') {
      steps { sh 'docker build -t myapp:1.0 .' }
    stage('Post-Build Scan (Trivy Image)') {
      steps { sh 'trivy image myapp:1.0 --exit-code 1 --severity HIGH,CRITICAL
              -f sarif -o trivy-image.sarif' }
    }
    stage('Archive Reports') {
      steps { archiveArtifacts artifacts: 'trivy-image.sarif', allowEmptyArchive: false }
    }
post {
  failure { echo 'Build blocked due to security issues. Fix and try again.' }
  }
}
```

HOW IT WORKS

1. Checkout code from repository
2. Scan source, dependencies, IaC and secrets (trivy fs .)
3. Build the container image
4. Scan the container image (trivy image)
5. If vulnerabilities found → pipeline fails (exit code 1)
6. If clean → archive reports for compliance
7. Only safe images move forward to deployment

KEY BENEFITS

- Shift-left security
- Stops vulnerable images early
- Automates security enforcement
- Saves time and protects production

19. TROUBLESHOOTING TRIVY

COMMON PROBLEMS AND HOW TO FIX THEM

1. VULNERABILITY DATABASE PROBLEMS

- Trivy uses vulnerability databases from the internet.
- Symptoms: Outdated data, update failures, or errors.
- Fixes:
  - Check your internet connection
  - Run: trivy --download-db-only
  - Verify cache directory permissions
  - Clear cache: trivy cache clean

2. SLOW SCANS

- Large images, big repositories or slow networks can slow scans down.
- Fixes:
  - Use --scanners to limit scans
  - Use --severity HIGH,CRITICAL
  - Enable cache: trivy image --cache-dir .trivycache
  - Scan smaller targets
  - Use a faster network or runner

3. UNEXPECTED FINDINGS

- You see vulnerabilities you didn't expect.
- Fixes:
  - Read the package details and CVE description
  - Check if it is in use
  - Verify the installed version
  - Confirm if a fix is available
  - Evaluate the real risk

4. MISSING VULNERABILITIES

- Expected issues are not showing up.
- Fixes:
  - Ensure DB is updated
  - Check if the package is detected
  - Verify the OS / package manager
  - Try scanning the image instead of filesystem (or vice versa)
  - Check for ignored vulnerabilities

5. REGISTRY AUTHENTICATION PROBLEMS

- Cannot scan private images or registries.
- Fixes:
  - Login first: docker login <registry>
  - Or use: trivy image --username USER -password PASS <image>
  - For CI: Use secure credentials or environment variables
  - Check token / permission scopes

6. IMAGE NOT FOUND

- Trivy cannot find the image.
- Fixes:
  - Verify the image name and tag
  - Check registry URL
  - Run: docker images
  - Pull the image first
  - Ensure you are authenticated

7. CI SCAN WORKS LOCALLY BUT FAILS IN PIPELINE

- Common causes: Missing permissions, no internet access, network restrictions, missing cache, wrong credentials.
- Fixes:
  - Compare local vs CI environment
  - Ensure DB update is allowed in CI
  - Verify credentials and secrets
  - Check firewall / proxy settings
  - Use verbose mode to see real error:

```
trivy image --debug <image>
```

  - Ensure runner has enough memory and disk

DEBUGGING WORKFLOW

1. REPRODUCE
Reproduce the issue in the same environment as CI or the failing system.

2. INSPECT
Collect details and logs. Use --debug for more information.

3. FIX
Fix the root cause. Update config, permissions, network, or commands.

4. RESCAN
Run Trivy again to confirm the issue is resolved.

BEST PRACTICES

- Always keep Trivy and databases updated
- Use specific tags instead of latest
- Scan early and often
- Fail builds on HIGH and CRITICAL issues
- Keep ignore rules minimal and documented
- Review results, don't ignore them blindly

TROUBLESHOOT SMARTER. SCAN SAFER. SHIP SECURE.

DEVSECOPS PROJECT

BUILD A SMALL END-TO-END SECURITY WORKFLOW

PROJECT ENVIRONMENT

- GIT REPOSITORY
- APPLICATION + DOCKERFILE
- TRIVY FILESYSTEM SCAN
- TRIVY SECRET SCAN
- TRIVY IaC SCAN
- DOCKER BUILD
- TRIVY IMAGE SCAN
- CI SECURITY GATE
- CONTAINER REGISTRY
- DEPLOYMENT

GOAL:
DELIVER SECURE CODE TO PRODUCTION

STUDENT TASKS

1. INSTALL AND CONFIGURE TRIVY
2. SCAN THE APPLICATION FILESYSTEM
3. SCAN FOR SECRETS
4. SCAN THE DOCKERFILE
5. SCAN TERRAFORM OR KUBERNETES YAML
6. BUILD A CONTAINER IMAGE
7. SCAN THE IMAGE
8. IDENTIFY HIGH AND CRITICAL VULNERABILITIES
9. REMEDIATE SELECTED FINDINGS
10. REBUILD AND RESCAN
11. INTEGRATE SCANNING INTO CI/CD
12. CONFIGURE THE PIPELINE TO STOP UNACCEPTABLE BUILDS
13. GENERATE A FINAL SECURITY REPORT

FINAL LEARNING OUTCOME

BY THE END OF THIS PROJECT, YOU WILL BUILD A REAL-WORLD SECURITY WORKFLOW USING TRIVY FROM YOUR CODE TO PRODUCTION, PREVENTING VULNERABILITIES BEFORE THEY REACH YOUR USERS.

20. COMPLETE BEGINNER DEVSECOPS PROJECT

BUILD A SMALL END-TO-END SECURITY WORKFLOW

PROJECT ENVIRONMENT

1. GIT REPOSITORY
2. APPLICATION + DOCKERFILE
3. TRIVY FILESYSTEM SCAN
4. TRIVY SECRET SCAN
5. TRIVY IaC SCAN (Terraform / Kubernetes YAML)
6. DOCKER BUILD
7. TRIVY IMAGE SCAN
8. CI SECURITY GATE (PASS / FAIL)
9. CONTAINER REGISTRY
10. DEPLOYMENT

SECURE EARLY. BUILD SAFELY. SHIP WITH CONFIDENCE.

STUDENT TASKS

1. INSTALL AND CONFIGURE TRIVY
2. SCAN THE APPLICATION FILESYSTEM
3. SCAN FOR SECRETS
4. SCAN THE DOCKERFILE
5. SCAN TERRAFORM OR KUBERNETES YAML
6. BUILD A CONTAINER IMAGE
7. SCAN THE IMAGE
8. IDENTIFY HIGH AND CRITICAL VULNERABILITIES
9. REMEDIATE SELECTED FINDINGS
10. REBUILD AND RESCAN
11. INTEGRATE SCANNING INTO CI/CD
12. CONFIGURE THE PIPELINE TO STOP UNACCEPTABLE BUILDS
13. GENERATE A FINAL SECURITY REPORT

FINAL OUTCOME

A PRACTICAL, END-TO-END SECURITY WORKFLOW USING TRIVY AT EVERY IMPORTANT STAGE OF THE DEVOPS LIFECYCLE.